Welcome

Digital and Computer Vision

Lesson 1: Session 2

Image Basics & Manipulation

Summary of the **previous** session: Introduction to Computer Vision

In the previous session we have covered :

What is Computer Vision? Applications in AI, robotics, and real-world industries.
Understanding digital image properties, formats, channels size, hape, pixels, RGB, Grayscale).
Overview of OpenCV. Installing OpenCV and setting up the Python environment.

Summary of the current session: Image Basics & Manipulation

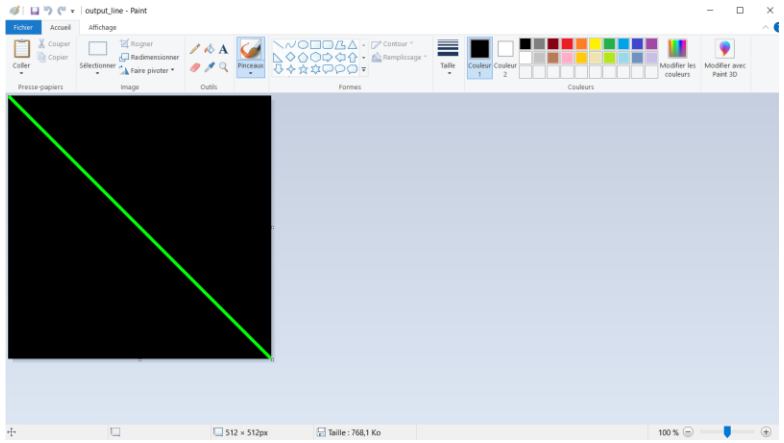
In this session we will covered the following:

Accessing pixel values.
Scaling, cropping, flipping and rotating images and translating images.
Perspective transformations and warping.
Changing Color-space (Grayscale, HSV, RGB).
Image arithmetic (adding, subtracting, blending images, bitwise operations).
Exercise: Try to find a way to extract colored object, for example, extract red, blue, or green objects.

Exercise 1 from previous session

Save on the disk the constructed image as jpg file using imsave :

```
import cv2
import numpy as np

image = np.zeros((512, 512, 3), np.uint8)
cv2.line(image, (0,0), (511, 511), (0, 255, 0), 5)
cv2.imshow("Image", image)

# save the green line as jpg image
# save the green line as bitmap image
cv2.imwrite("C:/Users/Konstantin/Documents/Avantgard/UMEF/DigitalCampus/Images/output_line.bmp", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

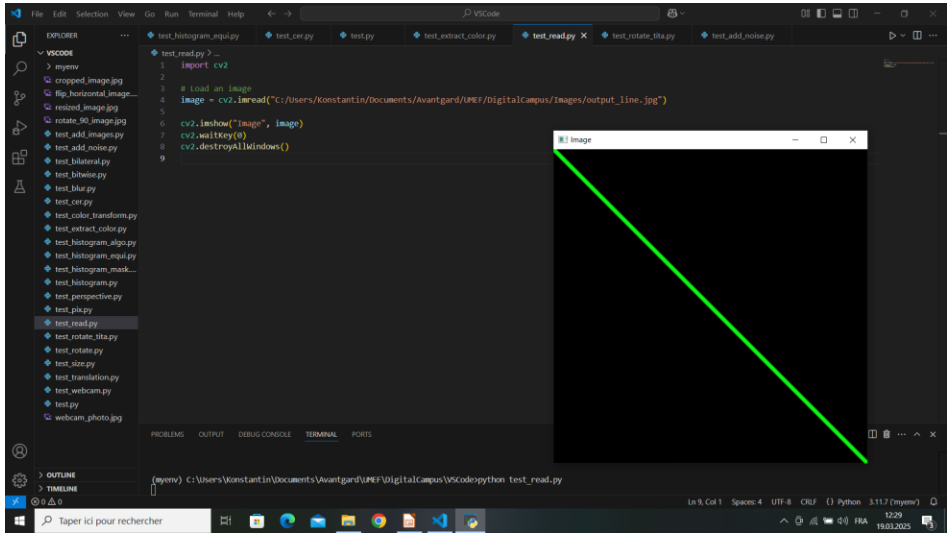


Exercise 2: Read this image with imread.

```
import cv2

# Load an image we are using here
image =
cv2.imread("C:/Users/Konstantin/Documents/Avantgard/UMEF/
DigitalCampus/Images/output_line.jpg")

cv2.imshow("Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

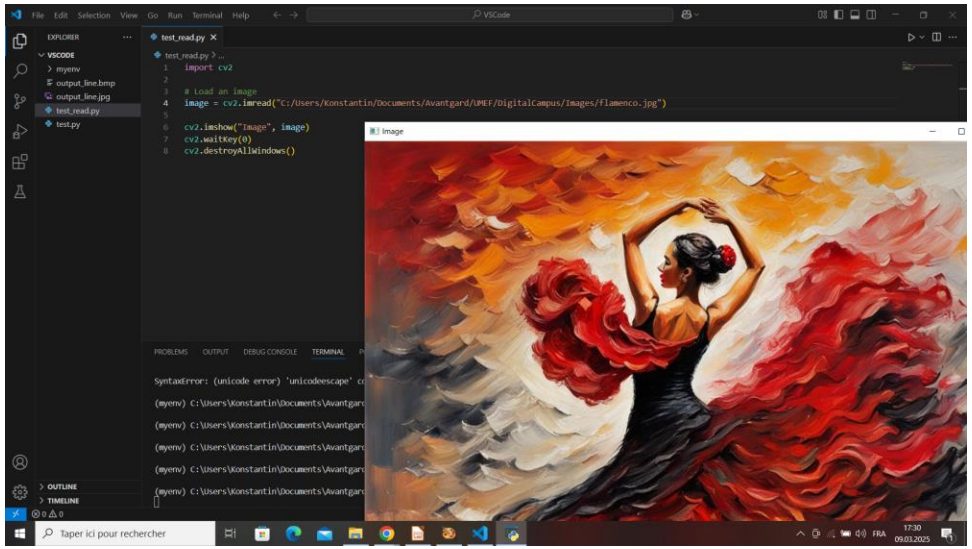
Save the file with a different name before run on the command line :
python test_read.py

NB : we don't need to reserve the memory for the image to be read



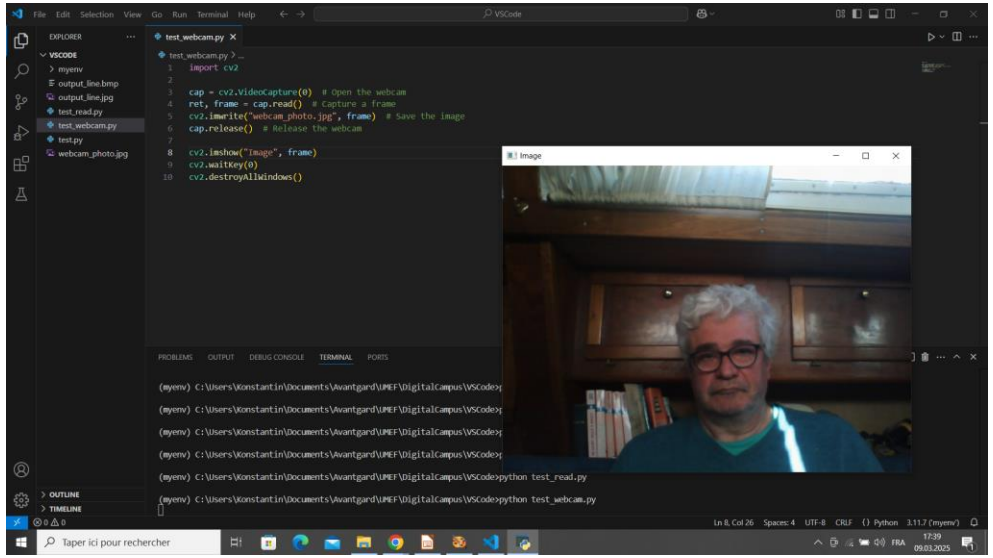
Exercice 3 : Read and show another image on your choice from the disk.

```
import cv2

# Load an image
image =
cv2.imread("C:/Users/Konstantin/Documents/Avantgard/UMEF/
DigitalCampus/Images/flamenco.jpg")

cv2.imshow("Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Exercice 4 : Save image from the web camera

```
import cv2

 # Open the webcam
cap = cv2.VideoCapture(0)
# Capture a frame ret is true if successfully captured
ret, frame = cap.read()

# Write the image
cv2.imwrite("../Images/webcam_photo.jpg", frame)

# Release the webcam
cap.release()

cv2.imshow("Image", frame)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Digital Image

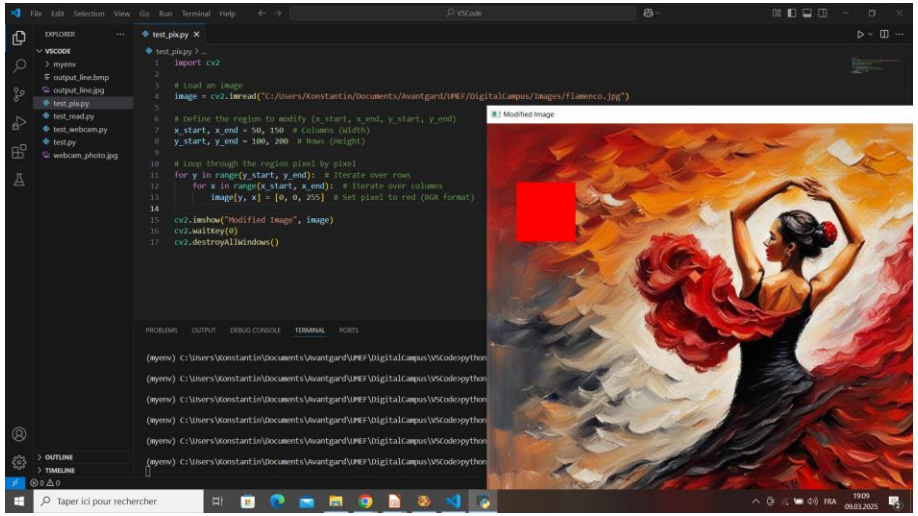
Pixel Values: An image is made up of pixels, which are the smallest units of information in an image. Each pixel has a value that represents its color and intensity. In RGB image values are usually represented as integers ranging from 0 to 255. 0 represents the absence of color (black), and 255 represents the maximum intensity of that color (full brightness).

Image Matrix: The computer reads the image as a matrix of numbers, where each element in the matrix corresponds to the pixel value at that location. For a color image, there are typically three matrices, one for each color channel.

$$f(x,y) = \begin{bmatrix} f(0,0) & f(0,1) & f(0,2) & \dots & f(0,N-1) \\ f(1,0) & f(1,1) & f(1,2) & \dots & f(1,N-1) \\ \vdots & \vdots & \vdots & & \vdots \\ f(M-1,0) & f(M-1,1) & f(M-1,2) & \dots & f(M-1,N-1) \end{bmatrix}$$

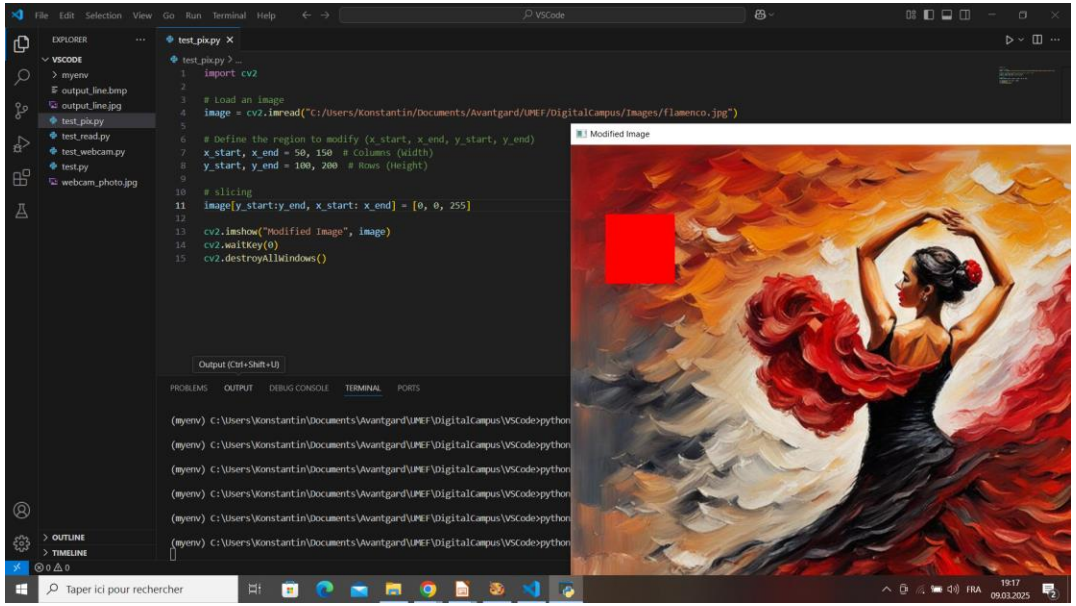
Accessing pixel values

```
import cv2

image = cv2.imread("../Images/flamenco.jpg")

# Define the region to modify (x_start, x_end, y_start, y_end)
x_start, x_end = 50, 150  # Columns (Width)
y_start, y_end = 100, 200  # Rows (Height)

# Loop through the region pixel by pixel
for y in range(y_start, y_end):  # Iterate over rows
    for x in range(x_start, x_end):  # Iterate over columns
        image[y, x] = [0, 0, 255]  # Set pixel to red (BGR format)

cv2.imshow("Modified Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



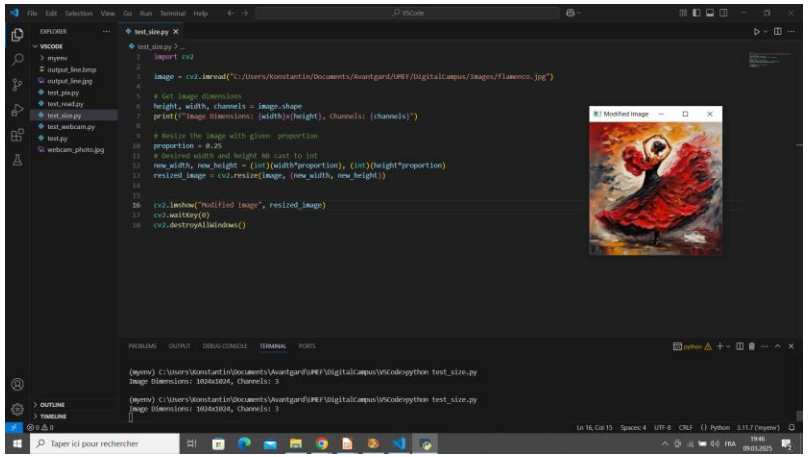
Using slicing

```
import cv2

image = cv2.imread("../Images/flamenco.jpg")

# Define the region to modify (x_start, x_end, y_start, y_end)
x_start, x_end = 50, 150  # Columns (Width)
y_start, y_end = 100, 200  # Rows (Height)

#  access region of interest (ROI) using slicing
image[y_start:y_end, x_start: x_end] = [0, 0, 255]

cv2.imshow("Modified Image", image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Resizing images

```
import cv2

image = cv2.imread("../Images/flamenco.jpg")

# Get image dimensions
height, width, channels = image.shape
print(f"Image Dimensions: {width}x{height}, Channels: {channels}")

# Resize the image with given  proportion
proportion = 0.25
# Desired width and height cast to int
new_width, new_height = (int)(width*proportion), (int)(height*proportion)
resized_image = cv2.resize(image, (new_width, new_height))
cv2.imshow("Modified Image", resized_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



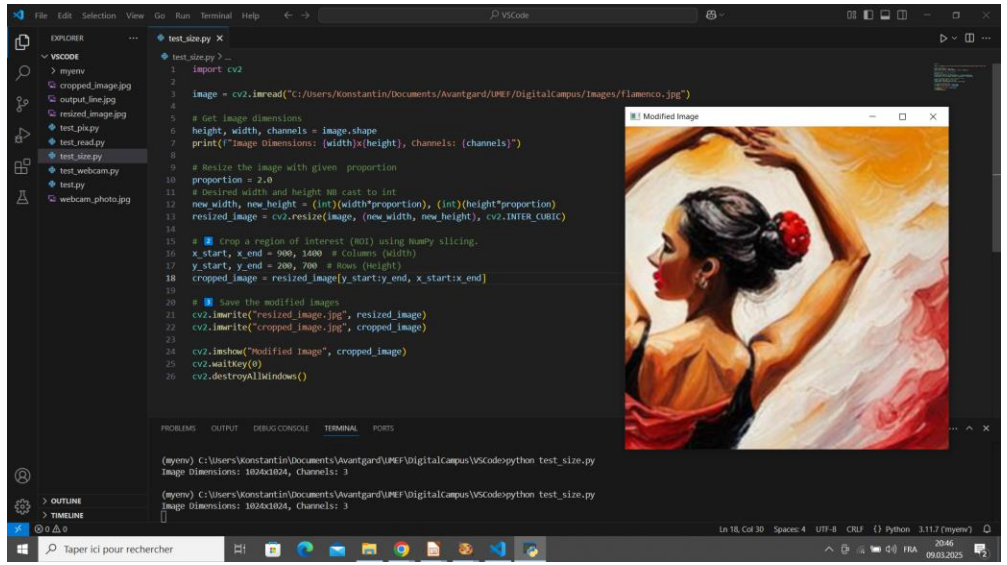
Resizing images - Algorithms

If you don't specify an interpolation method, OpenCV defaults to cv2.INTER_LINEAR.

Interpolation Method	Description	Best For
cv2.INTER_NEAREST	Nearest-neighbor without interpolation (fastest, low quality).	Simple graphics, pixel art
cv2.INTER_LINEAR	Bilinear interpolation of nearest 4 neighboring pixels (default, moderate quality).	General resizing
cv2.INTER_CUBIC	Bicubic interpolation (uses 16 pixels, higher quality but slower).	High-quality resizing
cv2.INTER_LANCZOS4	Lanczos interpolation (uses 8×8 pixel neighborhood, best quality).	High-resolution images
cv2.INTER_AREA	Area-based resampling (reduces noise, best for downscaling).	Shrinking images

Resizing and cropping

```
# Resize the image with given  proportion
proportion = 2.0
# Desired width and height NB cast to int
new_width, new_height = (int)(width*proportion), (int)(height*proportion)

resized_image = cv2.resize(image, (new_width, new_height),
cv2.INTER_CUBIC)

x_start, x_end = 900, 1400  # Columns (Width)
y_start, y_end = 200, 700  # Rows (Height)

# Crop a region of interest (ROI) using NumPy slicing.
cropped_image = resized_image[y_start:y_end, x_start:x_end]
```



Translation

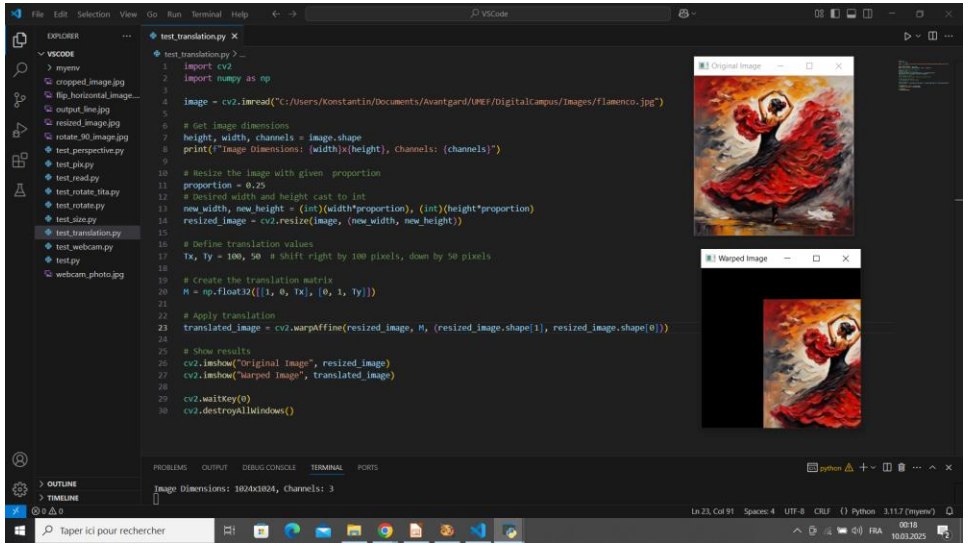
Translation is the shifting of an object's location.

$$x' = Mx$$

Where x' and x are the new and old positions of the image respectively
M is the translational matrix t_x and t_y are the shift,

$$M = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \end{bmatrix}$$

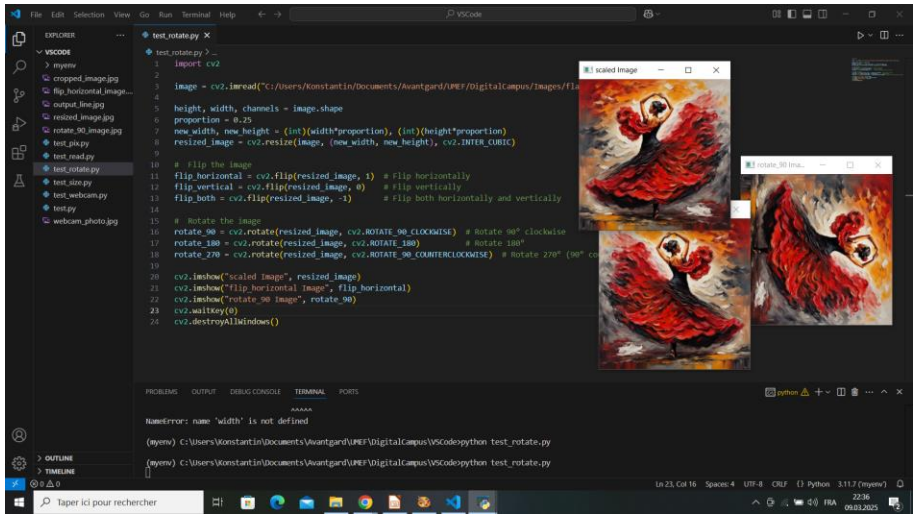
Translation - example




```
# Shift right by 100 pixels, down by 50 pixels
Tx, Ty = 100, 50

# Create the translation matrix
M = np.float32([[1, 0, Tx], [0, 1, Ty]])

# Apply translation
translated_image = cv2.warpAffine(image, M, (image.shape[1], image.shape[0]))
```

Flipping and rotating images at 90°

```
#  Flip the image
flip_horizontal = cv2.flip(resized_image, 1)  # Flip horizontally
flip_vertical = cv2.flip(resized_image, 0)    # Flip vertically
flip_both = cv2.flip(resized_image, -1)       # Flip both

#  Rotate the image
rotate_90 = cv2.rotate(resized_image, cv2.ROTATE_90_CLOCKWISE)
rotate_180 = cv2.rotate(resized_image, cv2.ROTATE_180)
```



Rotate an Image by a Custom Angle and scale by use of Rotation matrix

Rotation of an image for an angle tita is achieved by the transformation matrix of the form :

$$x' = Rx$$

$$\mathbf{R} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$$

But OpenCV provides scaled rotation with adjustable center of rotation so that you can rotate at any location you prefer. The modified transformation matrix is given by :

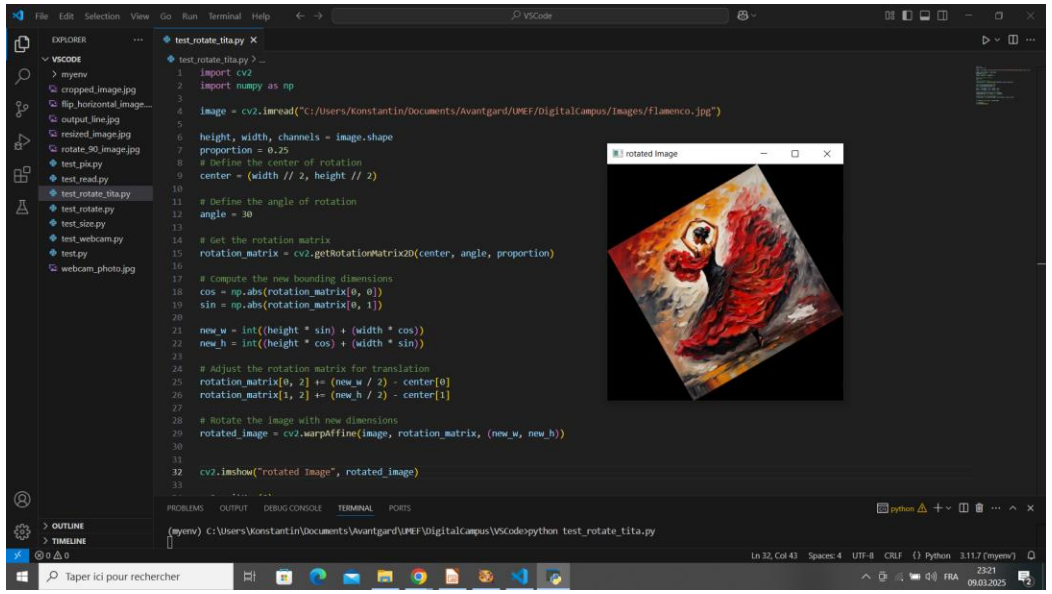
$$\begin{bmatrix} \alpha & \beta & (1-\alpha)\cdot center.x - \beta\cdot center.y \\ -\beta & \alpha & \beta\cdot center.x + (1-\alpha)\cdot center.y \end{bmatrix}$$

where :

$$\alpha = scale\cdot\cos\theta,$$
$$\beta = scale\cdot\sin\theta$$

Rotate an Image by a Custom Angle and scaling – example code

```
center = (width // 2, height // 2)  # Define the center of rotation
angle = 30 # Define the angle of rotation

# Get the rotation matrix
rotation_matrix = cv2.getRotationMatrix2D(center, angle, proportion)

# Compute the new bounding dimensions
cos = np.abs(rotation_matrix[0, 0])
sin = np.abs(rotation_matrix[0, 1])

new_w = int((height * sin) + (width * cos))
new_h = int((height * cos) + (width * sin))

# Adjust the rotation matrix for translation
rotation_matrix[0, 2] += (new_w / 2) - center[0]
rotation_matrix[1, 2] += (new_h / 2) - center[1]

# Rotate the image with new dimensions
rotated_image = cv2.warpAffine(image, rotation_matrix, (new_w, new_h))
```

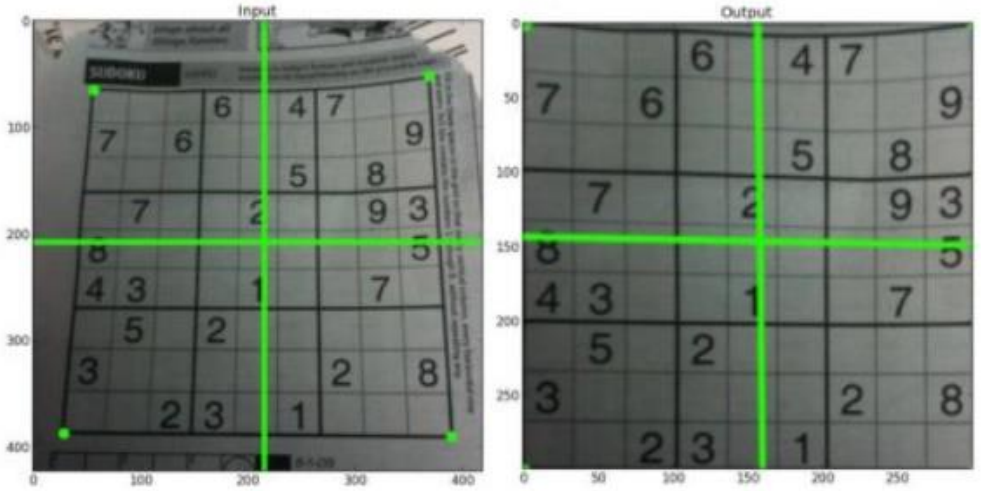


Perspective (projective) Transformation and Warping

Perspective transformation allows us to change the viewpoint of an image, making it look as if viewed from a different angle. Straight lines will remain straight even after the transformation. For perspective transformation, you need a 3x3 perspective transformation matrix H.:

$$x' = Hx$$

To find this transformation matrix, you need 4 points on the input image and corresponding points on the output image. Among these 4 points, 3 of them should not be collinear. Then the transformation matrix can be found by the function cv.getPerspectiveTransform. Then apply cv.warpPerspective with this 3x3 transformation matrix.

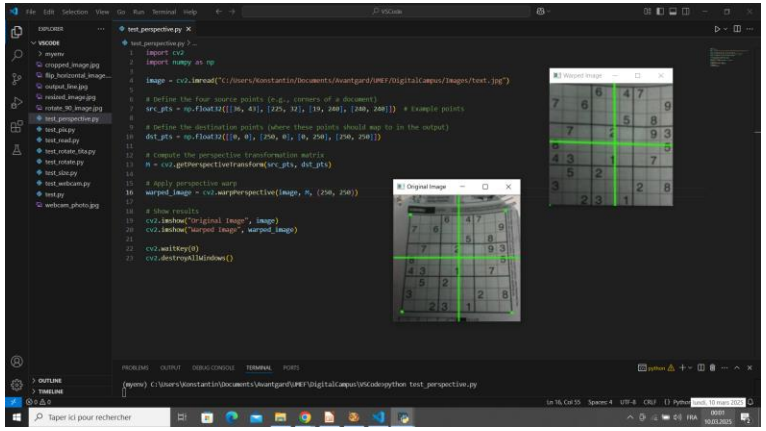


image

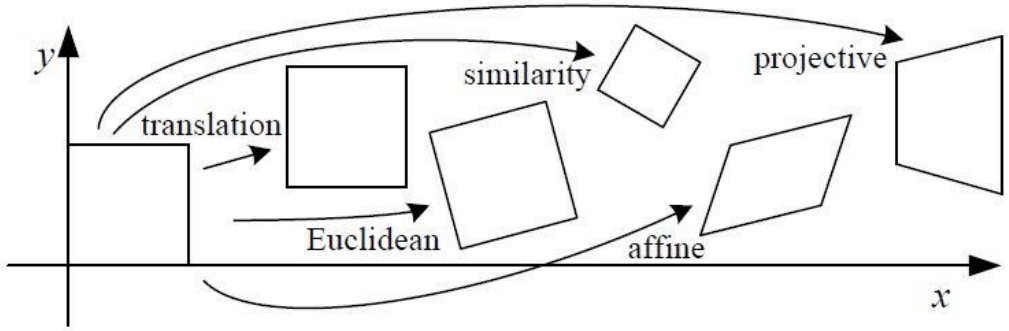
https://docs.opencv.org/4.x/da/d6e/tutorial_py_geometric_transformations.html

Perspective Transformation and Warping

```
image = cv2.imread("../Images/sudoku.png")

# Define the four source points
src_pts =np.float32([[56,65],[368,52],[28,387],[389,390]])

# Define the destination points
dst_pts = np.float32([[0,0],[300,0],[0,300],[300,300]])

# Compute the perspective transformation matrix
M = cv2.getPerspectiveTransform(src_pts, dst_pts)

# Apply perspective warp
warped_image = cv2.warpPerspective(image, M, (250, 250))
```



Summary of image transformations



Basic set of 2D planar transformations.

Transformation	Matrix	# DoF	Preserves	Icon
translation	$\begin{bmatrix} \mathbf{I} & \mathbf{t} \end{bmatrix}_{2\times 3}$	2	orientation	
rigid (Euclidean)	$\begin{bmatrix} \mathbf{R} & \mathbf{t} \end{bmatrix}_{2\times 3}$	3	lengths	
similarity	$\begin{bmatrix} s\mathbf{R} & \mathbf{t} \end{bmatrix}_{2\times 3}$	4	angles	
affine	$\begin{bmatrix} \mathbf{A} \end{bmatrix}_{2\times 3}$	6	parallelism	
projective	$\begin{bmatrix} \tilde{\mathbf{H}} \end{bmatrix}_{3\times 3}$	8	straight lines	

Szeliski, R. "Computer Vision: Algorithms and Applications." (https://szeliski.org/Book/)

Changing Color-space

There are more than 150 color-space conversion methods available in OpenCV. For this purpose we are using the function cv.cvtColor(src, f) where the first parameter is the image to transform and the second is the flag which give the type of color conversation,

Frequently used flags :
cv.COLOR_BGR2GRAY – BRG to gray scale conversation
cv.COLOR_BGR2HSV - BRG to HSV conversation
cv2.COLOR_BGR2RGB – BRG to RGB conversation

NB! OpenCV loads images in BGR (Blue-Green-Red) format by default instead of RGB (Red-Green-Blue) !

Changing Color-space example

```
import cv2

image = cv2.imread("../Images/flamenco.jpg")
height, width, channels = image.shape
proportion = 0.25
new_width, new_height = (int)(width*proportion), (int)(height*proportion)
resized_image = cv2.resize(image, (new_width, new_height))

# BGR to Grayscale
image_gray = cv2.cvtColor(resized_image, cv2.COLOR_BGR2GRAY)

# Show results
cv2.imshow("Original Image", resized_image)
cv2.imshow("Grayscale Image", image_gray)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

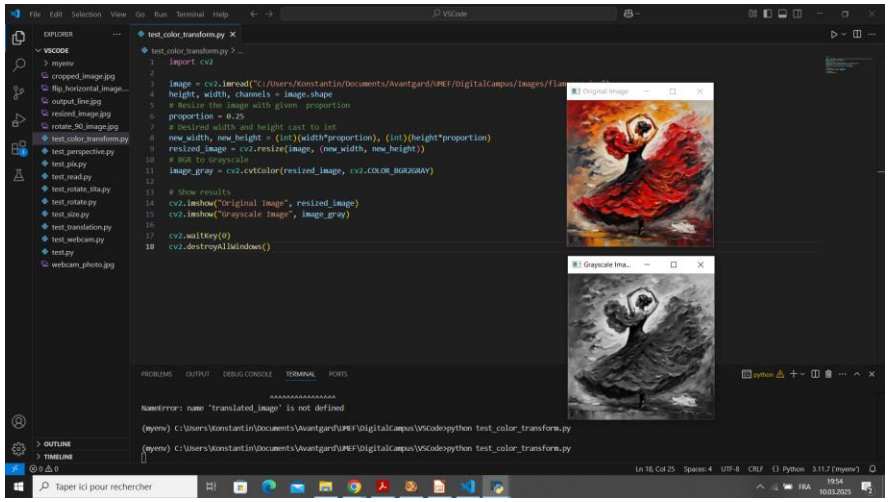


Image Addition

You can add two images with the OpenCV function, cv.add(), or by the numpy operation res = img1 + img2. Both images should be of same depth and type, or you can just add a scalar value.

NB! There is a difference between OpenCV addition and Numpy addition. OpenCV addition is a saturated operation while Numpy addition is a modulo operation (The modulo operator (%) returns the remainder after dividing one number by another)

For exaample :

```
>>> x = np.uint8([250])
>>> y = np.uint8([10])
>>> print( cv.add(x,y) ) # 250+10 = 260 => 255
>>> print( x+y )         # 250+10 = 260 % 256 = 4
```

Stick with OpenCV functions, because they will provide a better result.

Image Addition example

```
import cv2

image1 = cv2.imread("../Images/opencv-logo-white.png")
image2 = cv2.imread("../Images/ml.png")

# Define an empty result variable
added_image = None
if image1.shape != image2.shape:
    print("Resizing images to match dimensions...")
    image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

# Simple addition (clipping values between 0 and 255)
added_image = cv2.add(image1, image2)

cv2.imshow("Added Image", added_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

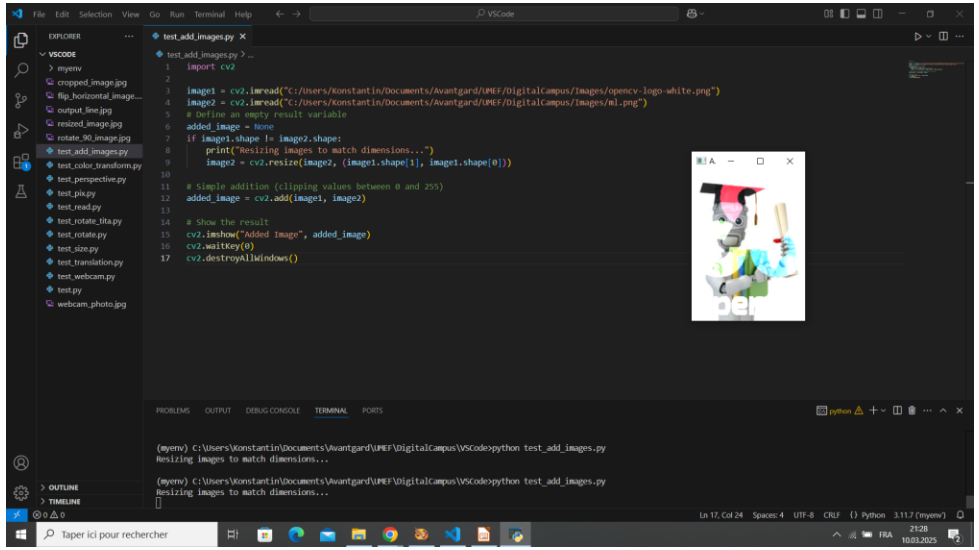


Image Blending

This is also image addition, but different weights are given to images in order to give a feeling of blending or transparency. Images are added as per the equation below:

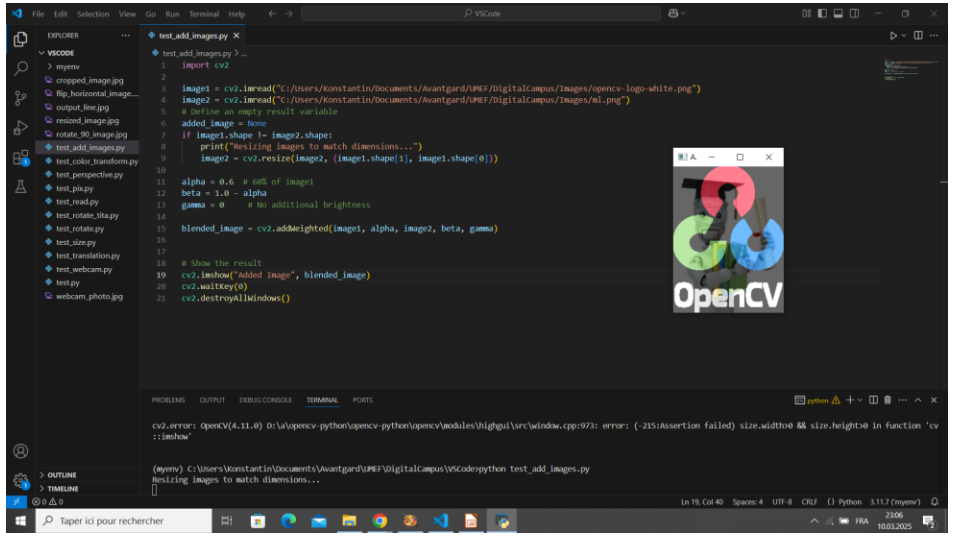
$$g(x) = (1 - \alpha)f_0(x) + \alpha f_1(x)$$

By varying alfa from 0.0 to 1.0, you can perform a cool transition between one image to another.

We use cv2.addWeighted(img1, α, img2, β, γ) for this purpose where gama add a constant brightness to make the result brighter or darker.

Image blending example

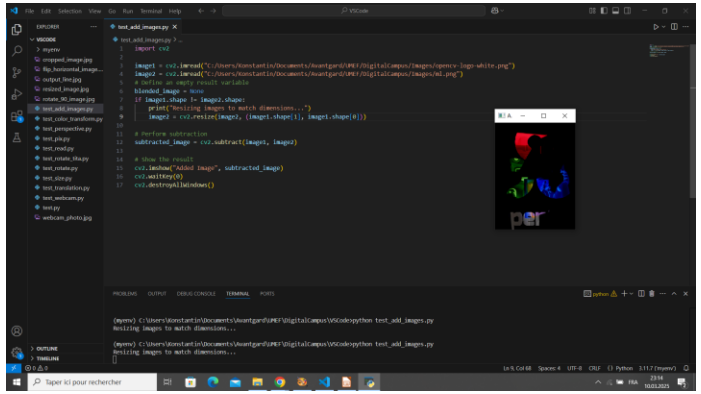
```
import cv2

image1 = cv2.imread("../Images/opencv-logo-white.png")
image2 = cv2.imread("../Images/ml.png")

# Define an empty result variable
blended_image = None
if image1.shape != image2.shape:
    print("Resizing images to match dimensions...")
    image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

alpha = 0.6  # 60% of image1
beta = 1.0 - alpha
gamma = 0     # No additional brightness
blended_image = cv2.addWeighted(image1, alpha, image2, beta, gamma)
cv2.imshow("Added Image", blended_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Substracting images example

You can subtract two images using OpenCV's cv2.subtract() function. This is useful for background removal, or edge enhancement.

```
import cv2

image1 = cv2.imread("../Images/opencv-logo-white.png")
image2 = cv2.imread("../Images/ml.png")

subtracted_image = None
if image1.shape != image2.shape:
    print("Resizing images to match dimensions...")
    image2 = cv2.resize(image2, (image1.shape[1], image1.shape[0]))

# Perform subtraction
subtracted_image = cv2.subtract(image1, image2)
cv2.imshow("Added Image", subtracted_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



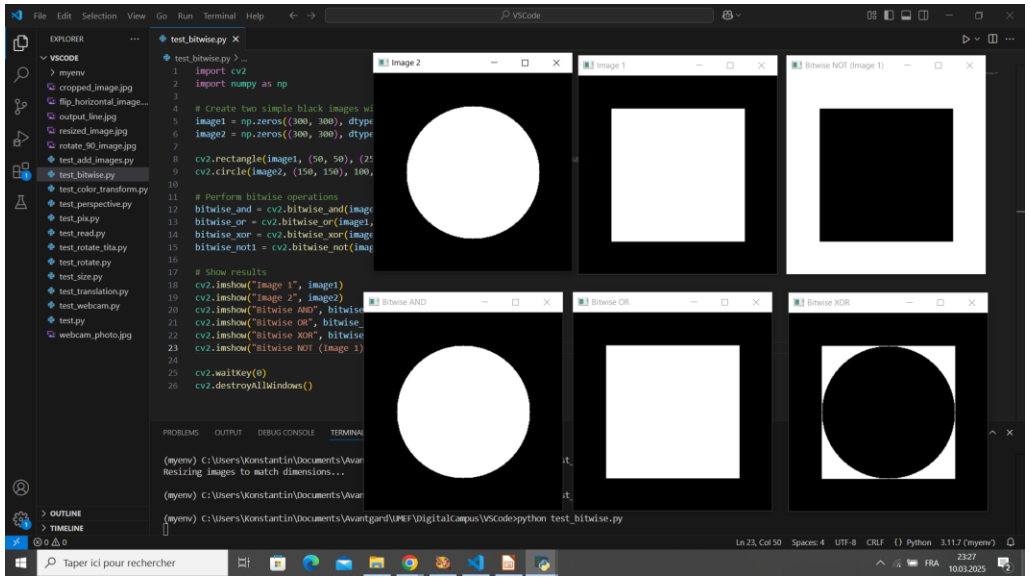
Bitwise Operations

Bitwise operations are useful for image masking, blending, and region extraction. OpenCV provides these functions:

cv2.bitwise_and() → Performs AND operation between two images.
cv2.bitwise_or() → Performs OR operation between two images.
cv2.bitwise_xor() → Performs XOR operation between two images.
cv2.bitwise_not() → Performs NOT operation (inverts colors).

Bitwise Operations

```
import cv2
import numpy as np

# Create two simple black images with white rectangles
image1 = np.zeros((300, 300), dtype=np.uint8)
image2 = np.zeros((300, 300), dtype=np.uint8)
cv2.rectangle(image1, (50, 50), (250, 250), 255, -1)  # White square in image1
cv2.circle(image2, (150, 150), 100, 255, -1)  # White circle in image2

# Keeps only overlapping white regions.
bitwise_and = cv2.bitwise_and(image1, image2)
# Merges both shapes (any white pixel remains white).
bitwise_or = cv2.bitwise_or(image1, image2)
#  Keeps non-overlapping white areas.
bitwise_xor = cv2.bitwise_xor(image1, image2)
#  Inverts black to white and vice versa.
bitwise_not1 = cv2.bitwise_not(image1)
```



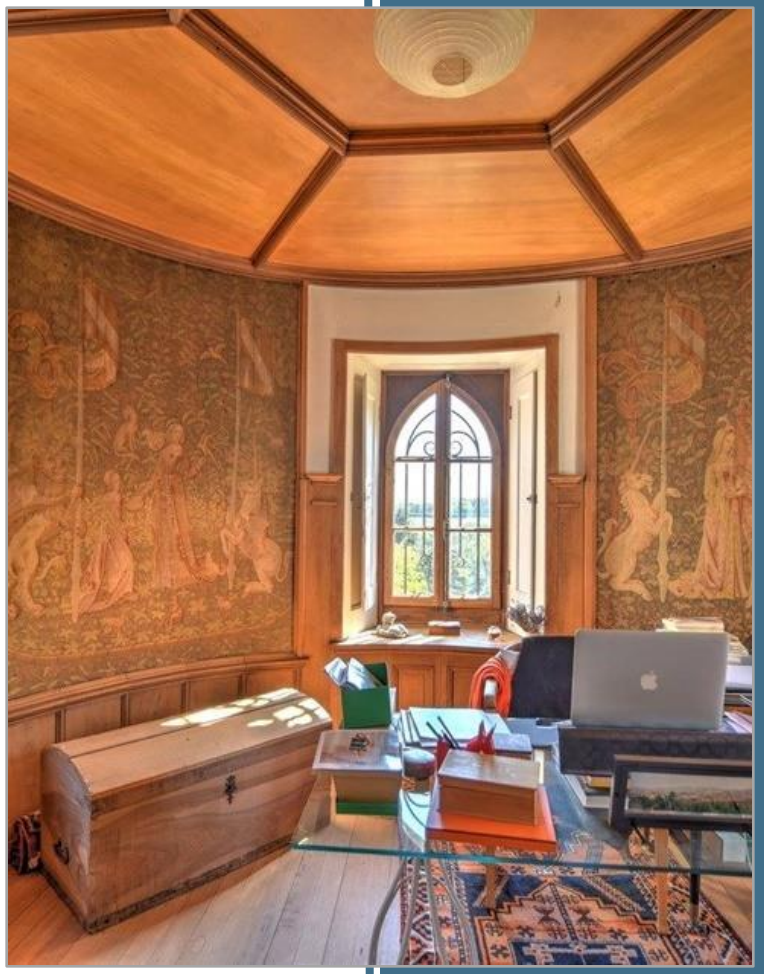
Exercise: Image Basics & Manipulation

Try to find an algorithm to extract the blue colored object from an still image, for example messi5.jpg

In this session we have covered:

Accessing pixel values.
Scaling, cropping, flipping and rotating images and translating images.
Perspective transformations and warping.
Changing Color-space (Grayscale, HSV, RGB).
Image arithmetic (adding, subtracting, blending images, bitwise operations).

In the next session we will covered:

Smoothing Techniques
Blurring and smoothing techniques (Gaussian, Median, Bilateral filters).
Removing noise using image filtering.
Histogram equalization for contrast enhancement.

THANKS !!!